Signal Lost (Integrity)

The Signal app is more than the sum of its protocols

Kien Tuong Truong[1], **Noemi Terzo**[2], Peter Schwabe[2,3], Kenny Paterson[1]

[1] ETH Zurich
[2] Max Planck Institute for Security and Privacy
[3] Radboud University

What’s Signal?

What’s Signal?

“The messaging app”

What’s Signal?

“The messaging app”

“The set of protocols”

What's Signal?

"The set of protocols"

Signal

The PQXDH Key Agreement Protocol

Signal

The X3DH Key Agreement Protocol

Signal

The Double Ratchet Algorithm

Signal

The Sesame Algorithm: Session Management for Asynchronous Message Encryption

Signal

Private Group Messaging

What's Signal?

"The set of protocols"

Signal

The PQXDH Key Agreement Protocol

Signal

Automated Verification for Secure Messaging Protocols and their Implementations: A Symbolic and Computational Approach

Nadim Kobeissi
INRIA Paris

Karthikeyan Bhargavan
INRIA Paris

Bruno Blanchet
INRIA Paris

chet Algorithm

Signal

The Sesame Algorithm: Session Management for Asynchronous Message Encryption

Signal

Private Group Messaging

What's Signal?

"The set of protocols"

Signal

The PQXDH Key Agreement Protocol

Signal

Automated Verification for Secure Messaging Prot
A Symbolic and Computation

Nadim Kobeissi
INRIA Paris

Karthikeyan Bhargavar
INRIA Paris

The Double Ratchet: Security Notions, Proofs, and Modularization for the Signal Protocol

Joël Alwen*
Wickr Inc.
jalwen@wickr.com

Sandro Coretti†
New York University
corettis@nyu.edu

Yevgeniy Dodis‡
New York University
dodis@cs.nyu.edu

February 21, 2020

Signal

The Sesame Algorithm: Session Management for Asynchronous Message Encryption

Signal

Private Group Messaging

What's Signal?

"The set of protocols"

Formal verification of the PQXDH Post-Quantum key agreement protocol for end-to-end secure messaging

Karthikeyan Bhargavan, *Cryspen*; Charlie Jacomme, *Inria Nancy Grand-Est, Université de Lorraine, LORIA, France*; Franziskus Kiefer, *Cryspen*; Rolfe Schmidt, *Signal Messenger*

Signal

Automated Verification for Secure Messaging Prot
A Symbolic and Computation

Nadim Kobeissi
INRIA Paris

Karthikeyan Bhargavar
INRIA Paris

A More Complete Analysis of the Signal Double Ratchet Algorithm

Alexander Bienstock[1(52)], Jaiden Fairoze[2], Sanjam Garg[2,3], Pratyay Mukherjee[4], and Srinivasan Raghuraman[5]

[1] New York University, New York, USA
abienstock@cs.nyu.edu
[2] UC Berkeley, Berkeley, USA
[3] NTT Research, Sunnyvale, USA
[4] Sicrikds Labs, Dallas, USA

Signal

The Sesame Algorithm: Session Management for Asynchronous Message Encryption

Revision 2, 2017-04-14 (PDF)

Moxie Marlinspike, Trevor Perrin (editor)

The Signal Private Group System and Anonymous Credentials Supporting Efficient Verifiable Encryption *

Melissa Chase
Microsoft Research
Redmond, Washington, USA
melissac@microsoft.com

Trevor Perrin
Signal Technology Foundation
USA
trevp@signal.org

Greg Zaverucha
Microsoft Research
Redmond, Washington, USA
gregz@microsoft.com

What's Signal?

"The set of protocols"

Formal verification of the PQXDH Post-Quantum

A Deniability Analysis of Signal's Initial Handshake PQXDH

Rune Fiedler
Technische Universität Darmstadt
Darmstadt, Germany
rune.fiedler@cryptoplexity.de

Christian Janson
Technische Universität Darmstadt
Darmstadt, Germany
christian.janson@cryptoplexity.de

Signal

Automated Verification for Secure Messaging Prot
A Symbolic and Computation

Nadim Kobeissi
INRIA Paris

Karthikeyan Bhargavar
INRIA Paris

Th

A More Complete Analysis of the Signal Double Ratchet Algorithm

and

A Formal Security Analysis of the Signal Messaging Protocol

(Extended abstract: full version available at [10])

Katriel Cohn-Gordon*, Cas Cremers*, Benjamin Dowling†, Luke Garratt*, Douglas Stebila‡
katriel.cohn-gordon@cs.ox.ac.uk
*University of Oxford, UK cas.cremers@cs.ox.ac.uk
luke.garratt@cs.ox.ac.uk
†Royal Holloway, University of London, UK benjamin.dowling@rhul.ac.uk
‡McMaster University, Canada stebilad@mcmaster.ca

Signal

The Sesame Algorithm: Session Management for Asynchronous Message Encryption

Revision 2, 2017-04-14 [PDF]

Maxie Marlinspike, Trevor Perrin (editor)

The Signal Private Group System and Anonymous Credentials Supporting Efficient Verifiable Encryption *

Melissa Chase
Microsoft Research
Redmond, Washington, USA
melissac@microsoft.com

Trevor Perrin
Signal Technology Foundation
USA
trevp@signal.org

Greg Zaverucha
Microsoft Research
Redmond, Washington, USA
greggz@microsoft.com

What's Signal?

"The set of protocols"

Security Analysis of Signal's **PQXDH** Handshake

Rune Fiedler[1](✉) and Felix Günther[2]

[1] Cryptoplexity, Technische Universität Darmstadt, Darmstadt, Germany
rune.fiedler@cryptoplexity.de
[2] IBM Research Europe – Zurich, Rüschlikon, Switzerland
mail@felixguenther.info

Signal Double Ratchet Algorithm

A Formal Security Analysis of the Signal Messaging Protocol

(Extended abstract: full version available at [10])

Katriel Cohn-Gordon*, Cas Cremers*, Benjamin Dowling†, Luke Garratt*, Douglas Stebila‡

*University of Oxford, UK
katriel.cohn-gordon@cs.ox.ac.uk
cas.cremers@cs.ox.ac.uk
luke.garratt@cs.ox.ac.uk

Automated Verification for Secure Messaging Prot
A Symbolic and Computation

Nadim Kobeissi
INRIA Paris

Karthikeyan Bhargavar
INRIA Paris

Signal

The Sesame Algorithm: Session Management for Asynchronous Message Encryption

Revision 2, 2017-04-14 [PDF]

Mexia Martinsplas, Trevor Perrin (editor)

WhatsUpp with Sender Keys? Analysis, Improvements and Security Proofs

David Balbás[1,2] *, Daniel Collins[3] **, and Phillip Gajland[4,5] ***

[1] IMDEA Software Institute, Spain
[2] Universidad Politécnica de Madrid, Spain
[3] EPFL, Switzerland
[4] Max Planck Institute for Security and Privacy, Germany
[5] Ruhr University Bochum, Germany

What's Signal?

"The set of protocols"

Security Analysis of Signal's **PQXDH** Handshake

Rune Fiedler[1(✉)] and Felix Günther[2]

[1] Cryptoplexity, Technische Universität Darmstadt, Darmstadt, Germany
rune.fiedler@cryptoplexity.de
[2] IBM Research Europe – Zurich, Rüschlikon, Switzerland

Comprehensive Deniability Analysis of Signal Handshake Protocols: X3DH, PQXDH to Fully Post-Quantum with

Shuichi Katsumata [1,2] *Guilhem Nie*
Thom Wiggers [1]
[1] *PQShield*
[2] *AIST*
[3] *Univ Rennes, CNRS, IRI*

Clone Detection in Secure Messaging: Improving Post-Compromise Security in Practice

Cas Cremers
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
cremers@cispa.saarland

Jaiden Fairoze
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
jfairoze@student.unimelb.edu.au

Benjamin Kiesl
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
benjamin.kiesl@cispa.saarland

Aurora Naska
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
skaunask@stud.uni-saarland.de

Formal Analysis of Session-Handling in Secure Messaging: Lifting Security from Sessions to Conversations

Cas Cremers
CISPA Helmholtz Center for Information Security

Charlie Jacomme*
Inria Paris, France

Aurora Naska
CISPA Helmholtz Center for Information Security

WhatsUpp with Sender Keys? Analysis, Improvements and Security Proofs

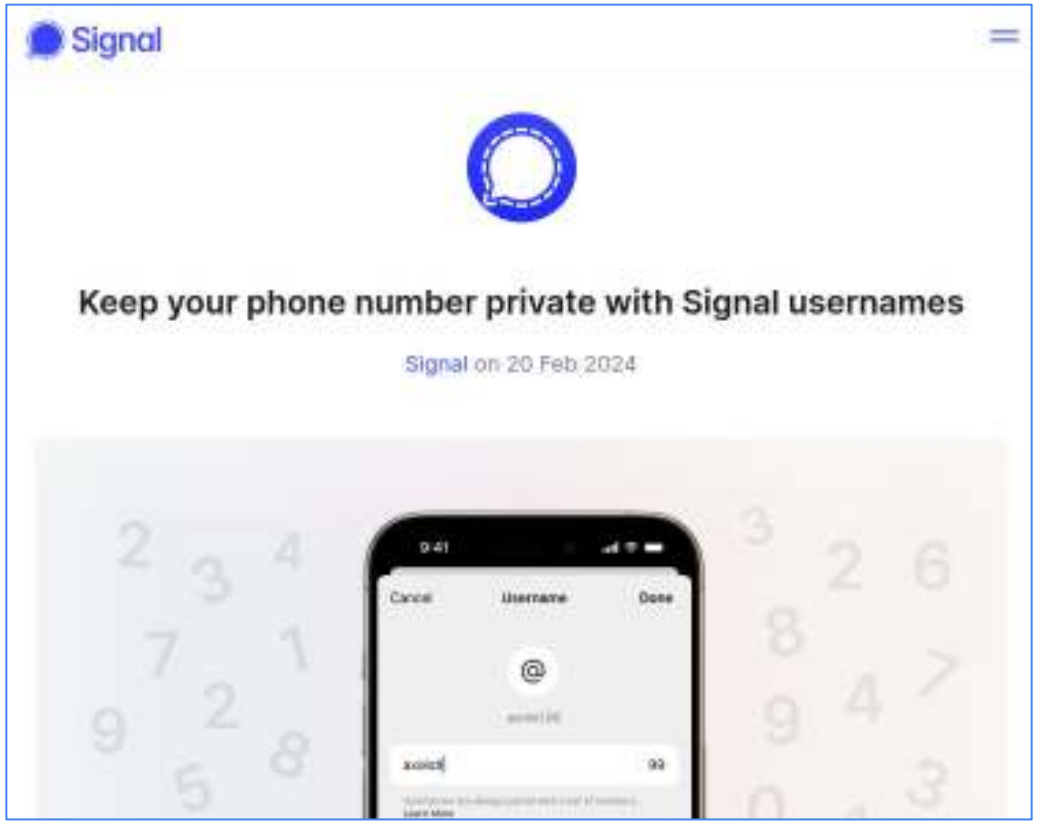
Analyzing Group Chat Encryption in MLS, Session, Signal, and Matrix

Joseph Jaeger and Akshaya Kumar

School of Cybersecurity and Privacy
Georgia Institute of Technology, Atlanta, Georgia, US
{josephjaeger,akshayakumar}@gatech.edu

What about the app?

Introduction of usernames in Signal



Introduction of usernames in Signal

Alice

ACI_A

PNI_A

Bob

ACI_B

PNI_B

Introduction of usernames in Signal

Alice

(pk_A^{ACI}, sk_A^{ACI})

$\mathbf{ACI_A}$

$\mathbf{PNI_A}$

(pk_B^{ACI}, sk_B^{ACI})

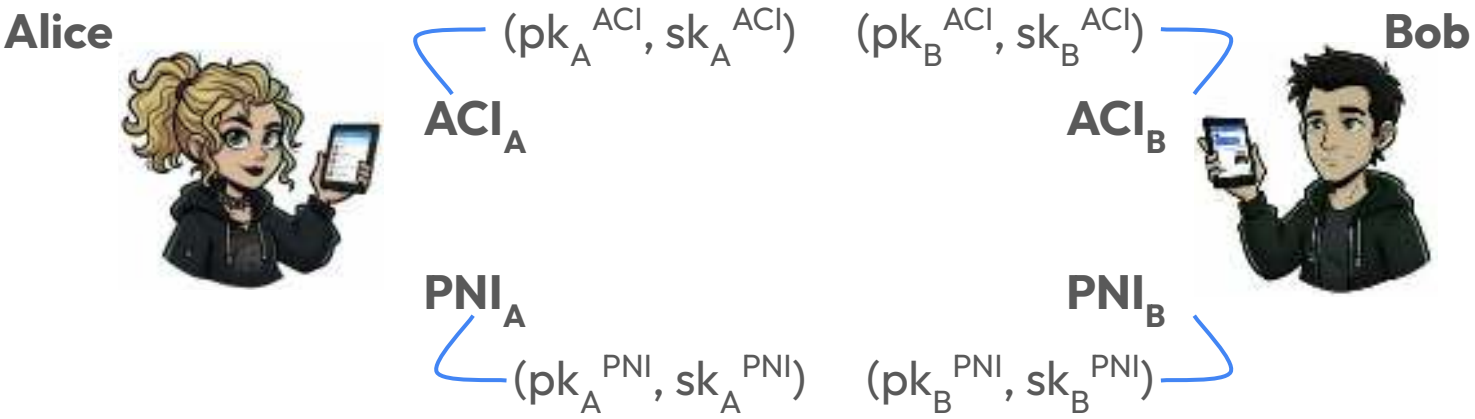
Bob

$\mathbf{ACI_B}$

$\mathbf{PNI_B}$

ACI: Account Identifier

Introduction of usernames in Signal



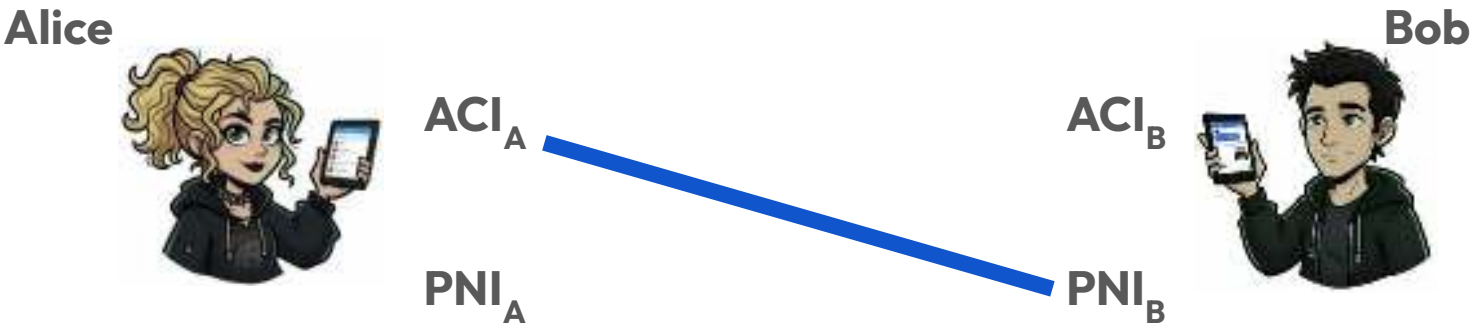
ACI: Account Identifier

PNI: Phone Number Identifier

ACIs, PNIs and sessions

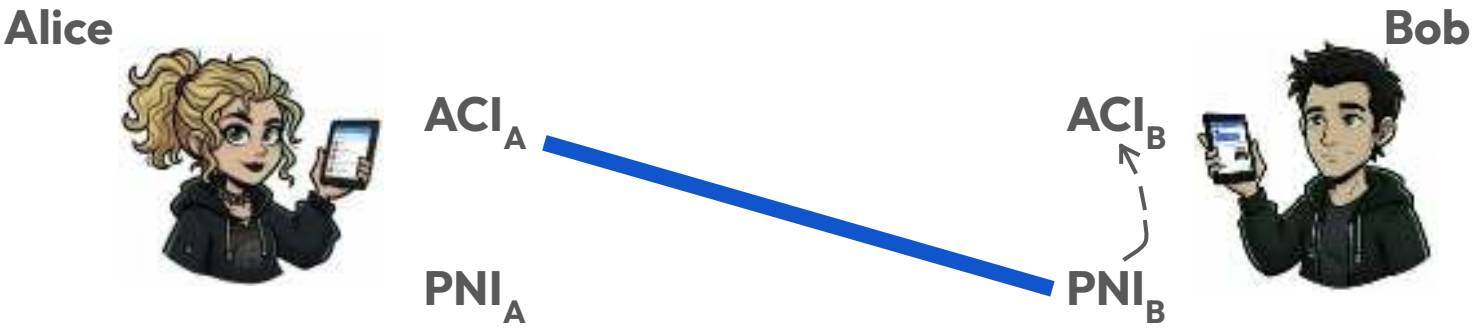
ACI_A – ACI_B session: by username

ACIs, PNIs and sessions



ACI_A – PNI_B session: by phone number

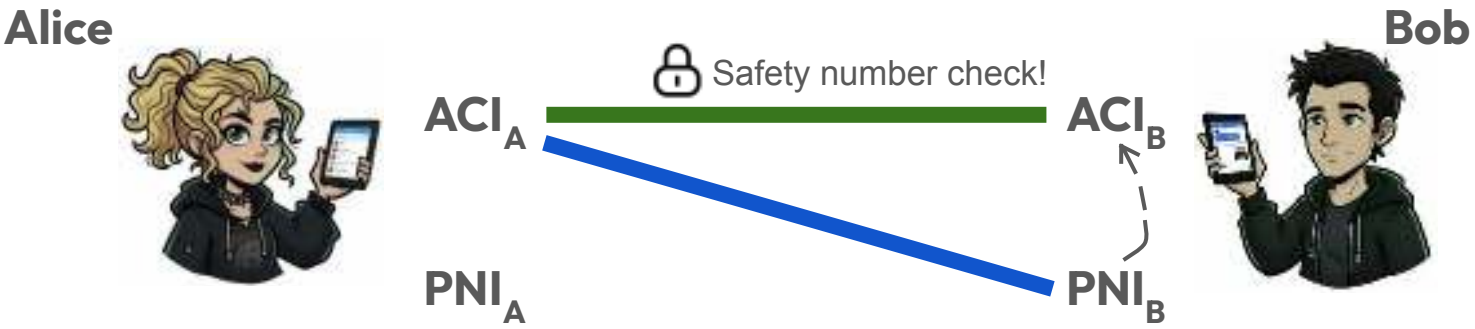
ACIs, PNIs and sessions



ACI_A – PNI_B session: by phone number

upgrade to ACI…

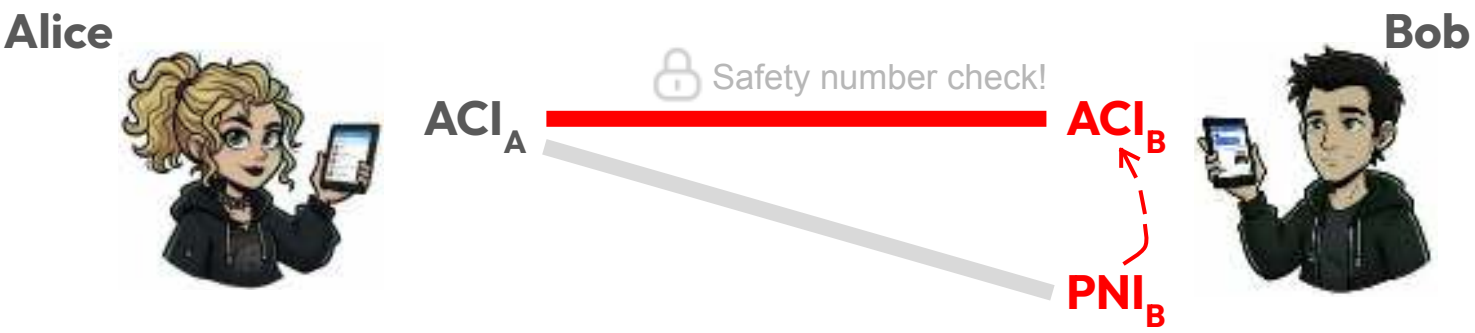
ACIs, PNIs and sessions



ACI$_A$ – PNI$_B$ session: by phone number

upgrade to ACI… new ACI$_A$ – ACI$_B$ session!

ACIs, PNIs and sessions



ACI_A – PNI_B session: by phone number

upgrade to ACI… new ACI_A – ACI_B session!

Upgrade to ACI

Alice

ACI_A

ACI_B

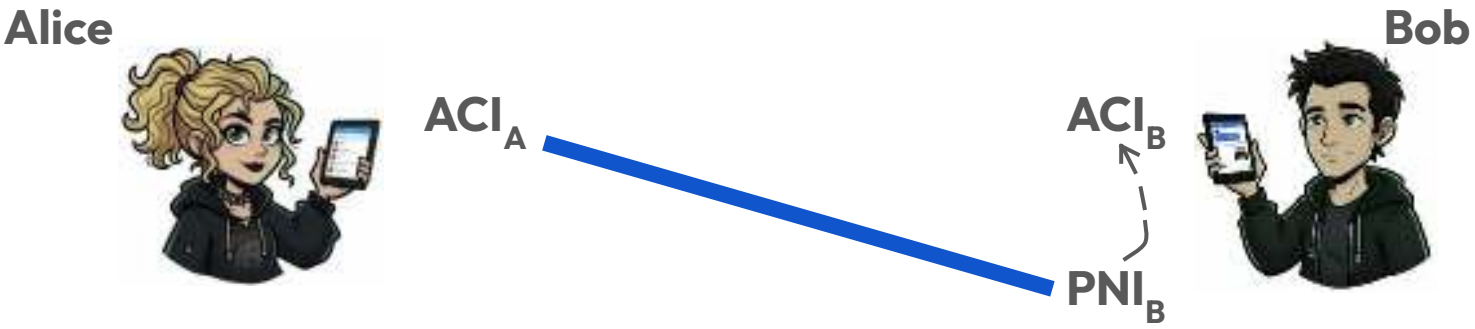
PNI_B

Bob

knows:

PNI_B, pk_B^{PNI}

Upgrade to ACI

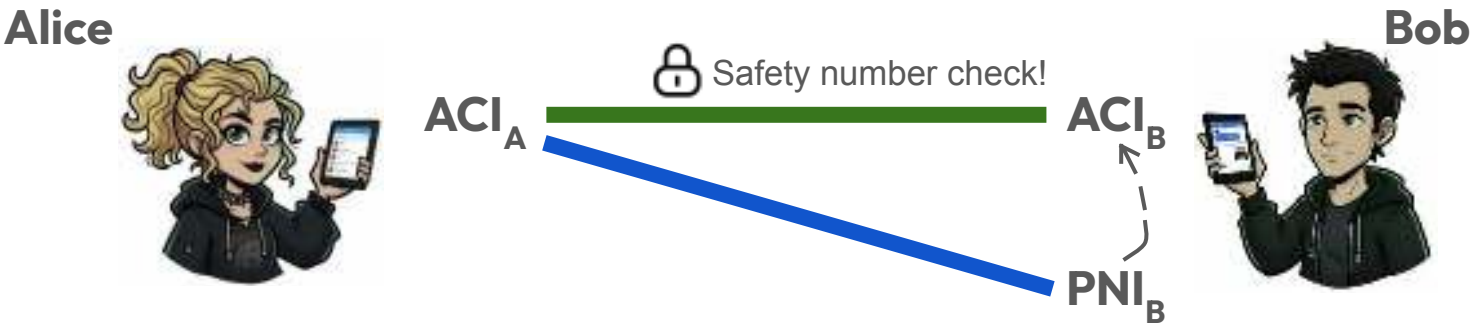


knows:

PNI_B, pk_B^{PNI}

signs pk_B^{ACI} with sk_B^{PNI}

Upgrade to ACI



knows:

PNI_B, pk_B^{PNI}

merges ACI_B, pk_B^{ACI}, PNI_B, pk_B^{PNI}

signs pk_B^{ACI} with sk_B^{PNI}

"PNI-to-ACI" protocol

Involved protocols: X3DH, Double Ratchet, Sesame

Upgrade to ACI

Alice

Safety number check!

ACI_A — ACI_B

PNI_B

Bob

knows:

PNI_B, pk_B^{PNI}

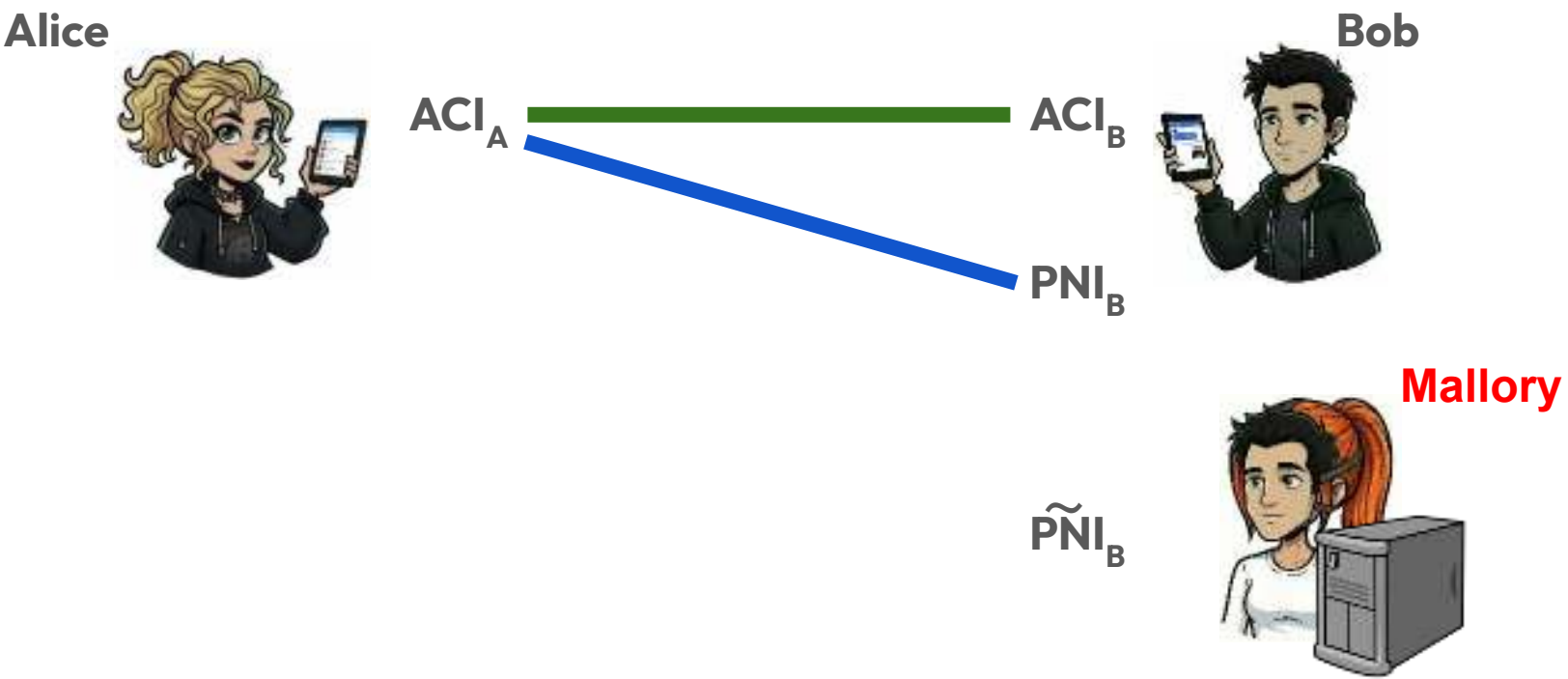
signs pk_B^{ACI} with sk_B^{PNI}

merges ACI_B, pk_B^{ACI}, PNI_B, pk_B^{PNI}

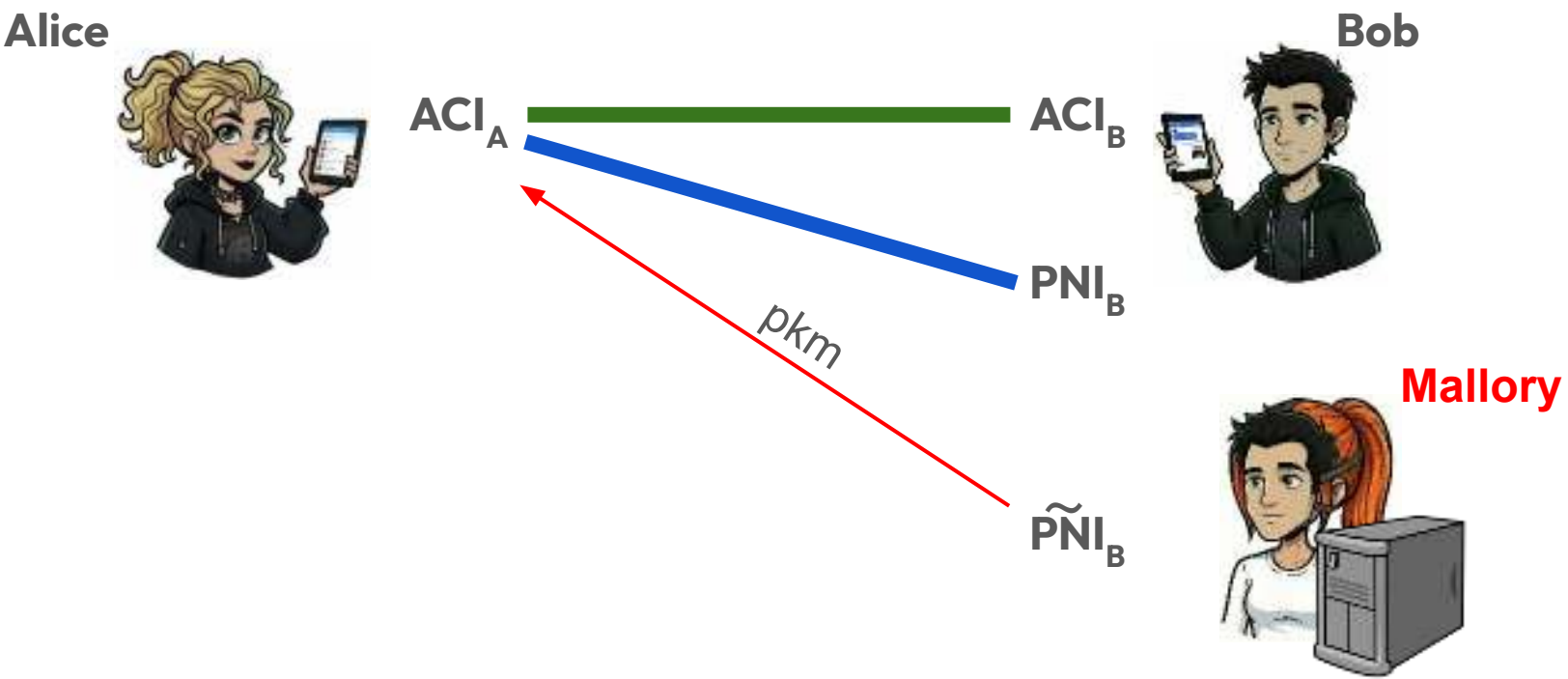
"PNI-to-ACI" protocol

Involved protocols: X3DH, Double Ratchet, Sesame

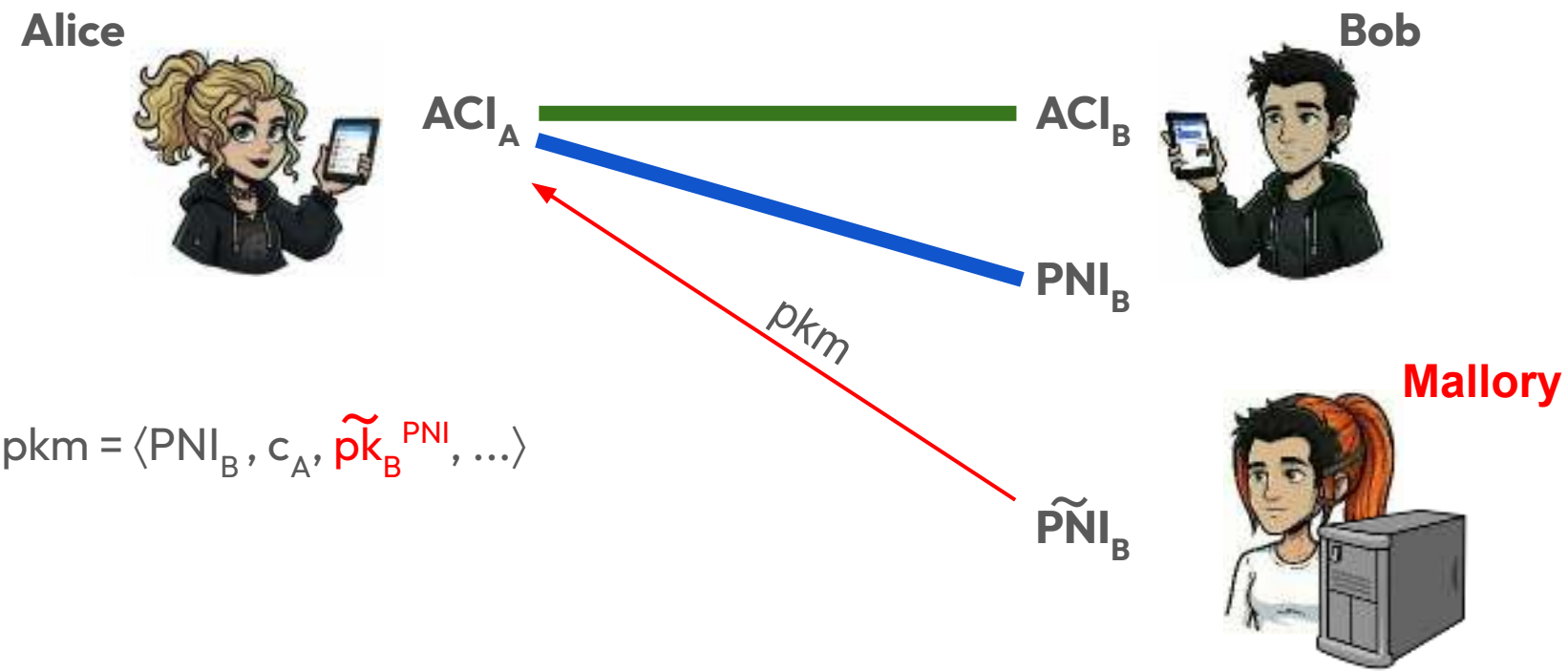
Attack Flow



Attack Flow

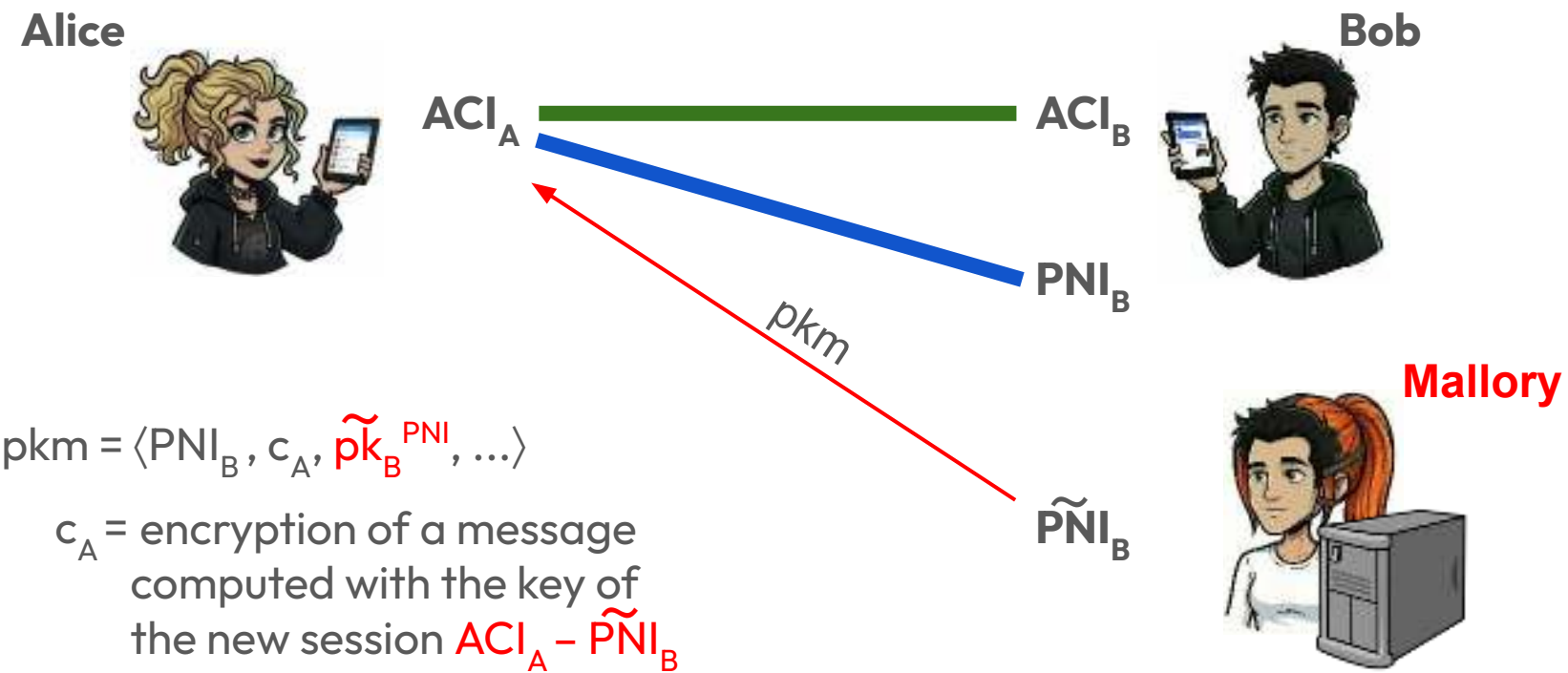


Attack Flow



$pkm = \langle PNI_B, c_A, \widetilde{pk}_B^{PNI}, \ldots \rangle$

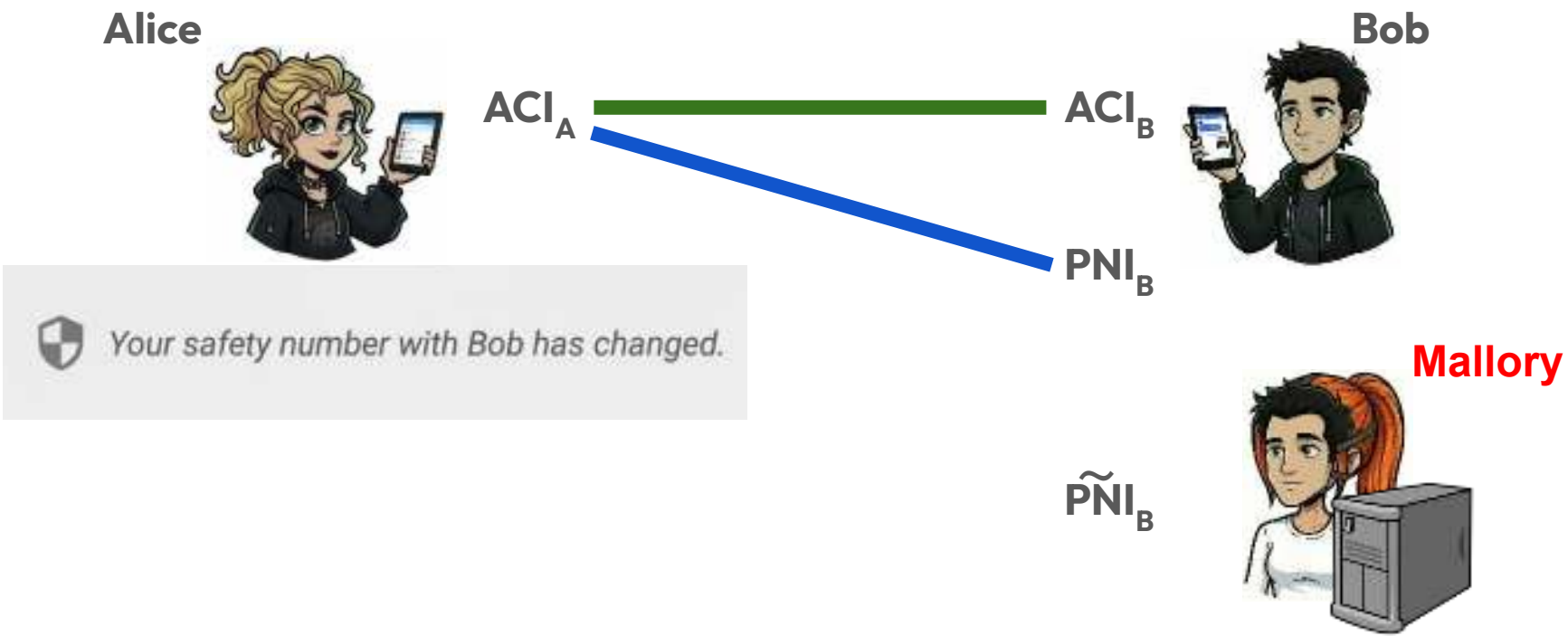
Attack Flow



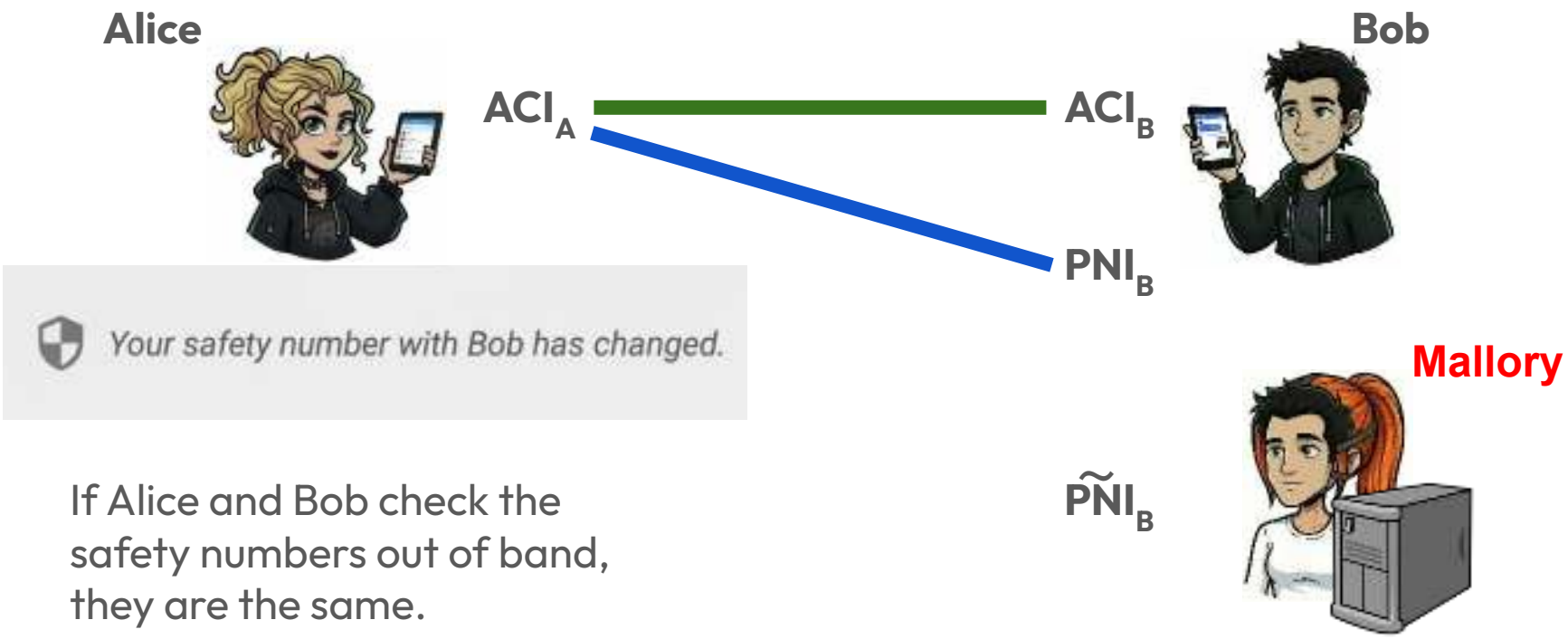
$pkm = \langle PNI_B, c_A, \widetilde{pk}_B^{PNI}, \ldots \rangle$

c_A = encryption of a message computed with the key of the new session $ACI_A - \widetilde{PNI}_B$

Attack Flow

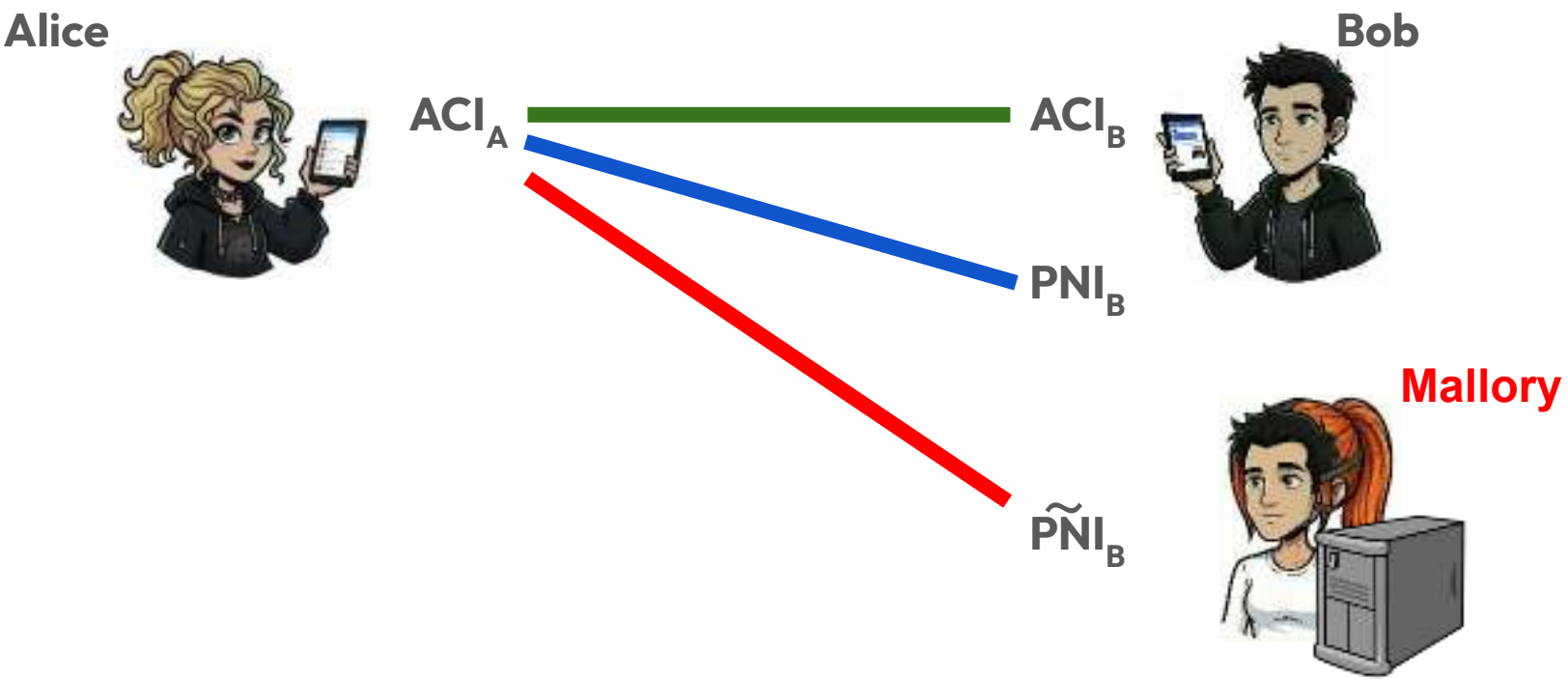

Alice
Bob
ACI_A
ACI_B
PNI_B
Your safety number with Bob has changed.
Mallory
$\widetilde{PNI}_B$

Attack Flow

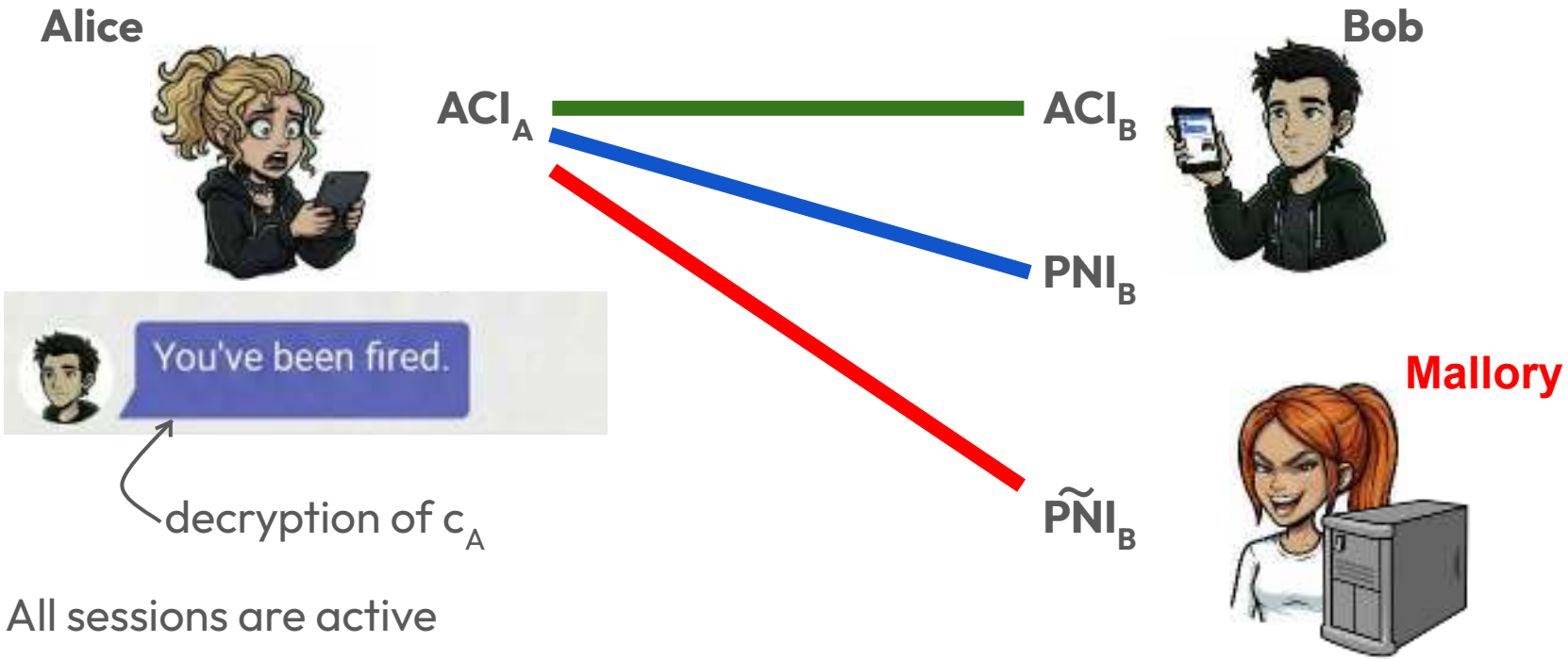


If Alice and Bob check the safety numbers out of band, they are the same.

Attack Flow



Attack Flow



All sessions are active

More about the attack

- The attack can be repeated injecting new messages

More about the attack

- The attack can be repeated injecting new messages
  - No “safety number changed” notification anymore

More about the attack

- The attack can be repeated injecting new messages
  - No “safety number changed” notification anymore
- The attack can be bidirectional

More about the attack

- The attack can be repeated injecting new messages
  - No “safety number changed” notification anymore
- The attack can be bidirectional

- Affected platforms
  - Android → up to v7.57.2
  - Desktop → up to v7.73.0

More about the attack

- The attack can be repeated injecting new messages
  - No “safety number changed” notification anymore
- The attack can be bidirectional

- Affected platforms
  - Android → up to v7.57.2
  - Desktop → up to v7.73.0

- Fixed in 1-2 days in September 2025 after responsible disclosure

More about the attack

- The attack can be repeated injecting new messages
  - No “safety number changed” notification anymore
- The attack can be bidirectional

- Affected platforms
  - Android → up to v7.57.2
  - Desktop → up to v7.73.0

- Fixed in 1-2 days in September 2025 after responsible disclosure
- Patch: do not accept messages coming from a PNI

More about the attack

- The attack can be repeated injecting new messages
  - No “safety number changed” notification anymore
- The attack can be bidirectional

- Affected platforms
  - Android → up to v7.57.2
  - Desktop → up to v7.73.0

- Fixed in 1-2 days in September 2025 after responsible disclosure
- Patch: do not accept messages coming from a PNI
- Devastating impact in real scenarios

Thanks for the attention!

noemi.terzo@mpi-sp.org
https://nterzo.sh

kitruong@ethz.ch
https://ktruong.dev

One more thing…

Signal Lost (Integrity): The Signal App is More than the Sum of its Protocols

Kien Tuong Truong, Noemi Terzo, Peter Schwabe, Kenneth Paterson

ETH Zurich,Max-Planck Institute for Security and Privacy

Hide abstract -

We present an attack against the integrity of conversations in Signal: we show that a malicious server can inject messages into a conversation between two honest users without them being aware of it. The attack does not require any key compromises. While the attack causes the honest users to receive a notification that their safety numbers have changed, those safety numbers remain consistent, so the attack cannot be detected by comparing them out-of-band. This attack naturally gives rise to a number of questions. How was this vulnerability introduced? How can such a vulnerability still be present after the extensive security analysis to which Signal's protocols have been subjected? What wider lessons can be drawn in order to prevent similar issues arising in the future? We answer these questions in detail in our talk.

Signal Lost (Integrity): The Signal App is More than the Sum of its Protocols

Kien Tuong Truong, Noemi Terzo, Peter Schwabe, Kenneth Paterson

ETH Zurich,Max-Planck Institute for Security and Privacy

Hide abstract -

We present an attack against the integrity of conversations in Signal: we show that a malicious server can inject messages into a conversation between two honest users without them being aware of it. The attack does not require any key compromises. While the attack causes the honest users to receive a notification that their safety numbers have changed, those safety numbers remain consistent, so the attack cannot be detected by comparing them out-of-band. This attack naturally gives rise to a number of questions. How was this vulnerability introduced? How can such a vulnerability still be present after the extensive security analysis to which Signal's protocols have been subjected? What wider lessons can be drawn in order to prevent similar issues arising in the future? We answer these questions in detail in our talk.

Signal Lost (Integrity): The Signal App is More than the Sum of its Protocols

Kien Tuong Truong, Noemi Terzo, Peter Schwabe, Kenneth Paterson

ETH Zurich,Max-Planck Institute for Security and Privacy

two attacks!

Hide abstract -

We present ~~an attack~~ against the integrity of conversations in Signal: we show that a malicious server can inject messages into a conversation between two honest users without them being aware of it. The attack does not require any key compromises. While the attack causes the honest users to receive a notification that their safety numbers have changed, those safety numbers remain consistent, so the attack cannot be detected by comparing them out-of-band. This attack naturally gives rise to a number of questions. How was this vulnerability introduced? How can such a vulnerability still be present after the extensive security analysis to which Signal's protocols have been subjected? What wider lessons can be drawn in order to prevent similar issues arising in the future? We answer these questions in detail in our talk.

Signal Lost (Integrity): The Signal App is More than the Sum of its Protocols

Kien Tuong Truong, Noemi Terzo, Peter Schwabe, Kenneth Paterson

ETH Zurich,Max-Planck Institute for Security and Privacy

two attacks!

Hide abstract -

We present ~~an attack~~ against the integrity of conversations in Signal: we show that a malicious server can inject messages into a conversation between two honest users without them being aware of it. The attack does not require any key compromises. While the attack causes the honest users to receive a notification that their safety numbers have changed, those safety numbers remain consistent, so the attack cannot be detected by comparing them out-of-band. This attack naturally gives rise to a number of questions. How was this vulnerability introduced? How can such a vulnerability still be present after the extensive security analysis to which Signal's protocols have been subjected? What wider lessons can be drawn in order to prevent similar issues arising in the future? We answer these questions in detail in our talk.

Signal Lost (Integrity): The Signal App is More than the Sum of its

Kien Tuong Tr enneth Paterson

ETH Privacy

two att

We present ons in Signal: we
show that a malici conversation
between two hone it. The attack does
not require any ke es the honest users
to receive a notific hanged, those
safety numbers re be detected by
comparing them c rise to a number of
questions. How w an such a
vulnerability still b nalysis to which
Signal's protocols ons can be drawn
in order to prevent e answer these
questions in detail in our talk.

Signal Lost (Integrity): The Signal App is More than the Sum of its Protocols

Kien Tuong Truong, Noemi Terzo, Peter Schwabe, Kenneth Paterson

ETH Zurich,Max-Planck Institute for Security and Privacy

two attacks!

Hide abstract -

We present ~~an attack~~ against the integrity of conversations in Signal: we show that a malicious server can inject messages into a conversation between two honest users without them being aware of it. The attack does not require any key compromises. While the attack causes the honest users to receive a notification that their safety numbers have changed, those safety numbers remain consistent, so the attack cannot be detected by comparing them out-of-band. This attack naturally gives rise to a number of questions. How was this vulnerability introduced? How can such a vulnerability still be present after the extensive security analysis to which Signal's protocols have been subjected? What wider lessons can be drawn in order to prevent similar issues arising in the future? We answer these questions in detail in our talk.

Signal Lost (Integrity): The Signal App is More than the Sum of its Protocols

Kien Tuong Truong, Noemi Terzo, Peter Schwabe, Kenneth Paterson

ETH Zurich,Max-Planck Institute for Security and Privacy

two attacks!

Hide abstract -

We present ~~an attack~~ against the integrity of conversations in Signal: we show that a malicious server can inject messages into a conversation between two honest users without them being aware of it. The attack does not require any key compromises. ~~While the attack causes the honest users to receive a notification that their safety numbers have changed, those safety numbers remain consistent~~, so the attack cannot be detected ~~by comparing them out-of-band~~. This attack naturally gives rise to a number of questions. How was this vulnerability introduced? How can such a vulnerability still be present after the extensive security analysis to which Signal's protocols have been subjected? What wider lessons can be drawn in order to prevent similar issues arising in the future? We answer these questions in detail in our talk.

Users don't receive any notification!

Attack No. 2

Our attack leverages two issues:

Attack No. 2

Our attack leverages two issues:

- One issue in libsignal, regarding **authentication in Sealed Sender**

Attack No. 2

Our attack leverages two issues:

- One issue in libsignal, regarding **authentication in Sealed Sender**
- One issue on Android, regarding **message validation**

Attack No. 2

Our attack leverages two issues:

- One issue in libsignal, regarding **authentication in Sealed Sender**
- One issue on Android, regarding **message validation**

Their combination allows **undetectable message injection** from a malicious server

Signal Sealed Sender

From: Alice
To: Bob

Signal Sealed Sender

pk_A

Wants to wrap msg m

pk_B

Signal Sealed Sender

pk_A

Wants to wrap msg m

$PKE.Enc(pk_B, msg=pk_A||ACI_A||cert)$

pk_B

Signal Sealed Sender

pk_A

Reveals the sender identity to Bob

pk_B

$PKE.Enc(pk_B, msg=pk_A||ACI_A||cert)$

Wants to wrap msg m

Signal Sealed Sender

The Signal server signs (pk_A, ACI_A) to certify the association

pk_A

Reveals the sender identity to Bob

$PKE.Enc(pk_B, msg=pk_A||ACI_A||cert)$

pk_B

Wants to wrap msg m

Signal Sealed Sender

The Signal server signs (pk_A, ACI_A) to certify the association

pk_A

Reveals the sender identity to Bob

pk_B

$PKE.Enc(pk_B, msg=pk_A||ACI_A||cert)$

$AuthPKE.Enc(pk_B, sender_pk=pk_A, msg=m)$

Wants to wrap msg m

Signal Sealed Sender

The Signal server signs (pk_A, ACI_A) to certify the association

pk_A

Reveals the sender identity to Bob

pk_B

$$\text{PKE.Enc}(pk_B, \text{msg}=pk_A||ACI_A||\text{cert})$$

$$\text{AuthPKE.Enc}(pk_B, \text{sender_pk}=pk_A, \text{msg}=m)$$

Wraps m and ensures it was sent by Alice

Wants to wrap msg m

Signal Sealed Sender

The Signal server signs (pk_A, ACI_A) to certify the association

pk_A

Reveals the sender identity to Bob

pk_B

$$\text{PKE.Enc}(pk_B, \text{msg}=pk_A||ACI_A||\text{cert})$$

$$\text{AuthPKE.Enc}(pk_B, \text{sender_pk}=pk_A, \text{msg}=m)$$

Wraps m and ensures it was sent by Alice

Wants to wrap msg m

Unfortunately, cert doesn't provide much if the server is malicious...

Issue 1: Sealed Sender does not authenticate

pk_M

Wants to wrap msg m

$PKE.Enc(pk_B, msg=pk_M||ACI_A||cert')$

pk_B

Issue 1: Sealed Sender does not authenticate

pk_M

Wants to wrap msg m

$PKE.Enc(pk_B, msg=pk_M||ACI_A||cert')$

$AuthPKE.Enc(pk_B, sender_pk=pk_M, msg=m)$

pk_B

Issue 1: Sealed Sender does not authenticate

pk_M

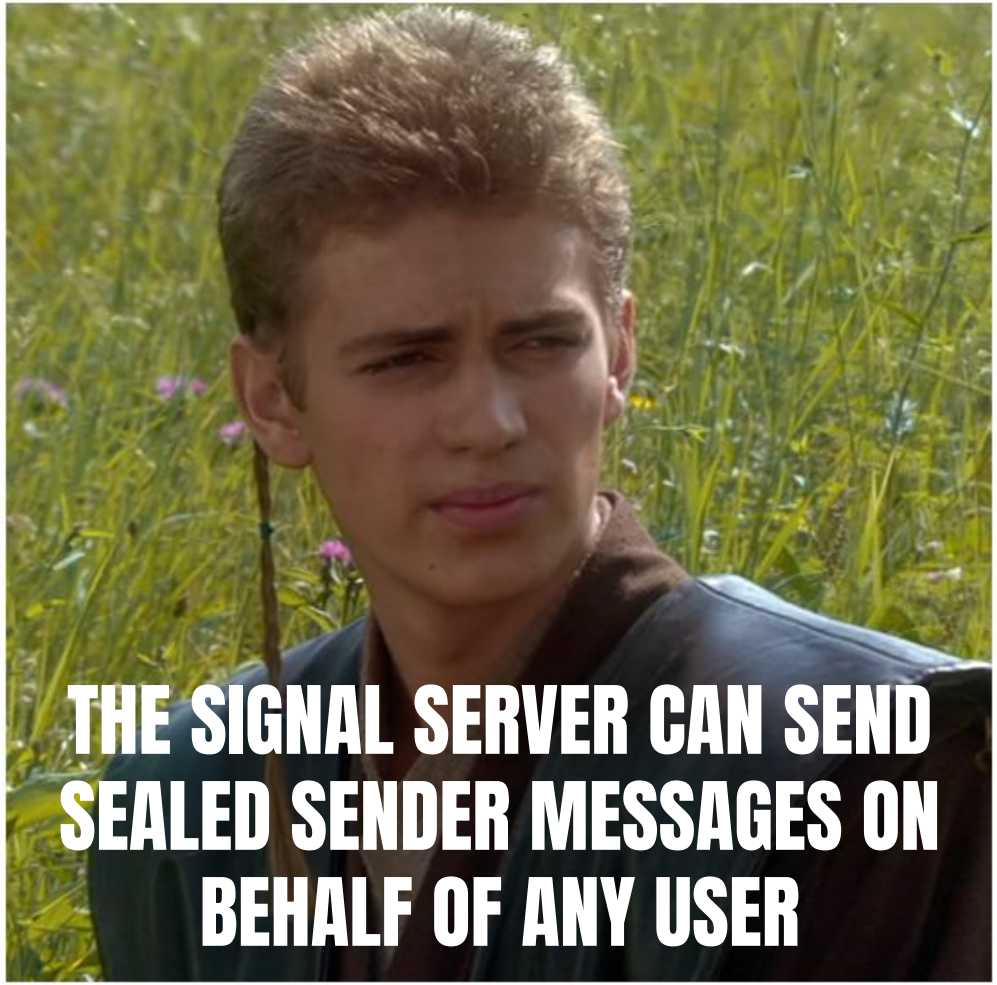
Wants to wrap msg m

$PKE.Enc(pk_B, msg=pk_M||ACI_A||cert')$

$AuthPKE.Enc(pk_B, sender_pk=pk_M, msg=m)$

pk_B

Bob **could** check whether pk_M is the key he knows for ACI_A, but libsignal doesn't do so.

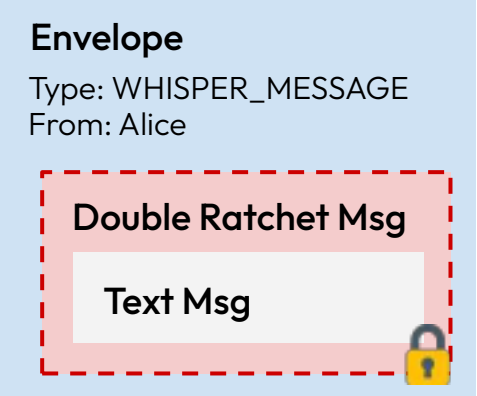

THE SIGNAL SERVER CAN SEND
SEALED SENDER MESSAGES ON
BEHALF OF ANY USER

BUT THERE'S ANOTHER LAYER OF
AUTHENTICATION PROVIDED BY THE DOUBLE
RATCHET AND SENDER KEYS, RIGHT?

...RIGHT?

Structure of a Message

Structure of a Message



Structure of a Message

Envelope

Type: WHISPER_MESSAGE
From: Alice

Double Ratchet Msg

Text Msg

Envelope

Type: GROUP_MESSAGE
From: Alice

Sender Keys Msg

Text Msg

Structure of a Message

Envelope
Type: WHISPER_MESSAGE
From: Alice

Double Ratchet Msg

Text Msg

Envelope
Type: PLAINTEXT_CONTENT
From: Alice

Plaintext Msg

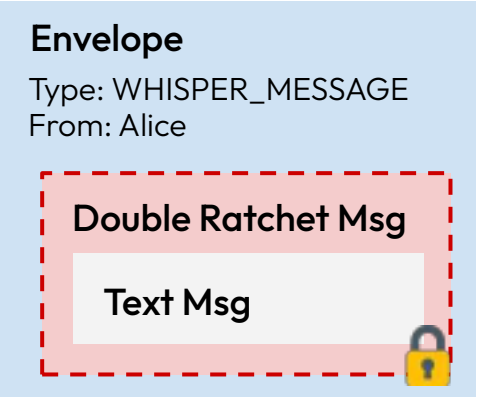
Text Msg

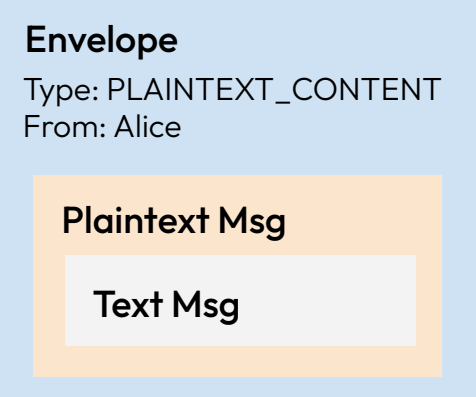
Envelope
Type: GROUP_MESSAGE
From: Alice

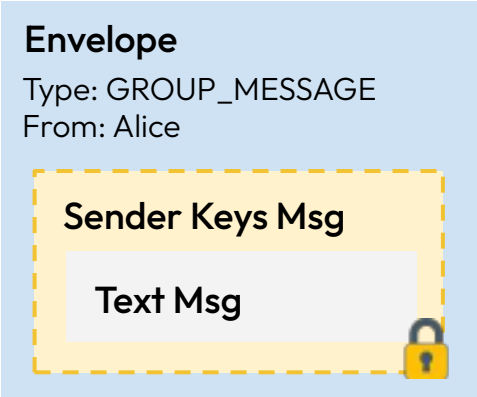
Sender Keys Msg

Text Msg

Structure of a Message

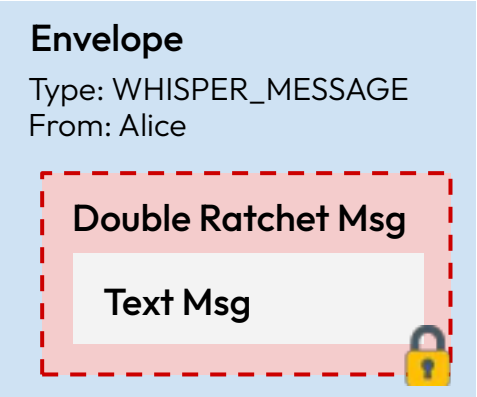


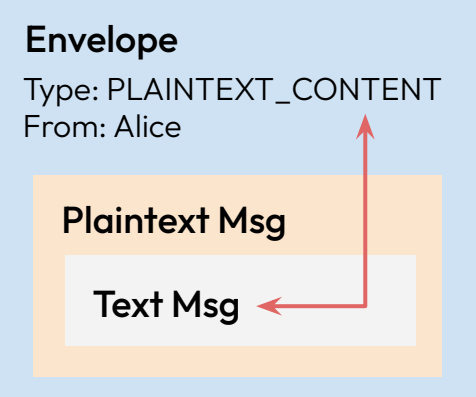


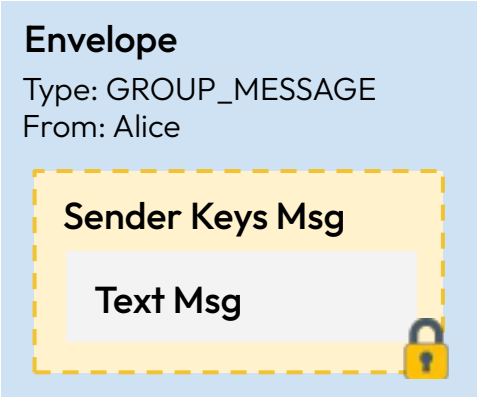


Since there is no authentication, Plaintext messages containing Text messages cannot be accepted. Usually they only carry DecryptionError messages.

Structure of a Message

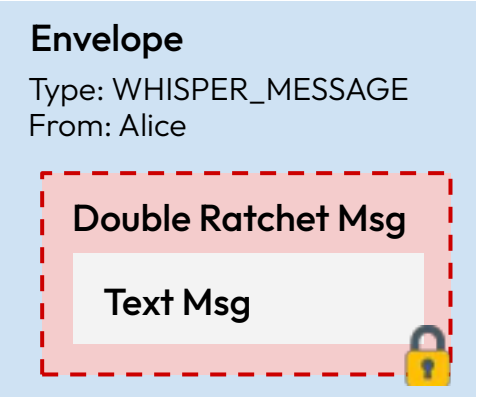


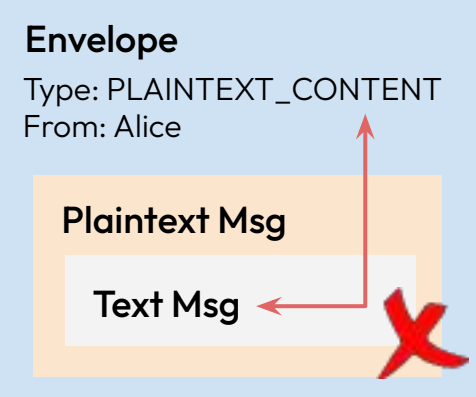


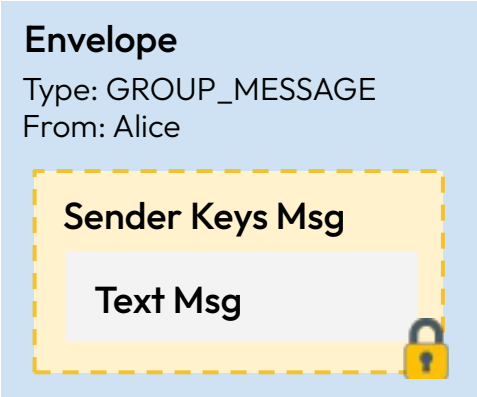


Since there is no authentication, Plaintext messages containing Text messages cannot be accepted. Usually they only carry DecryptionError messages.

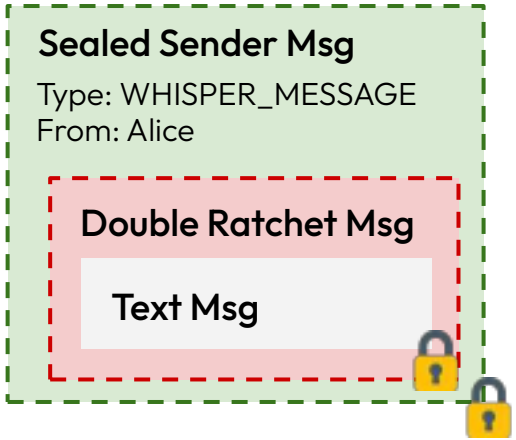
Structure of a Message

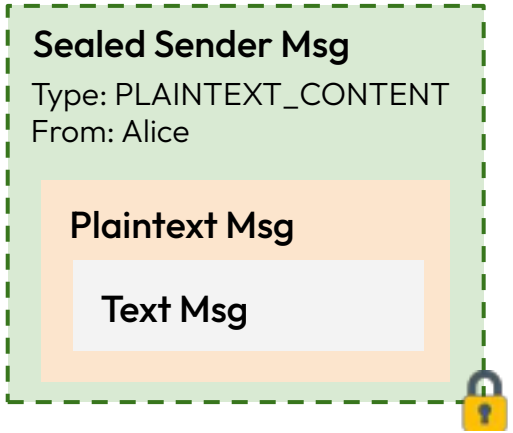


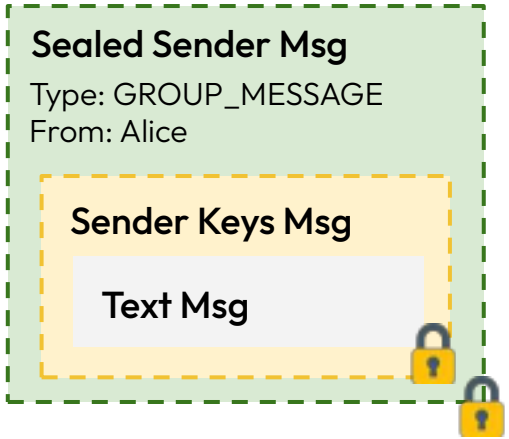


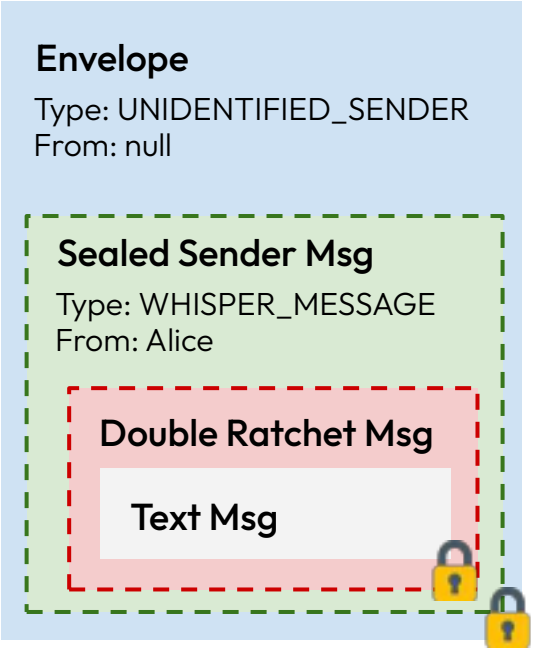


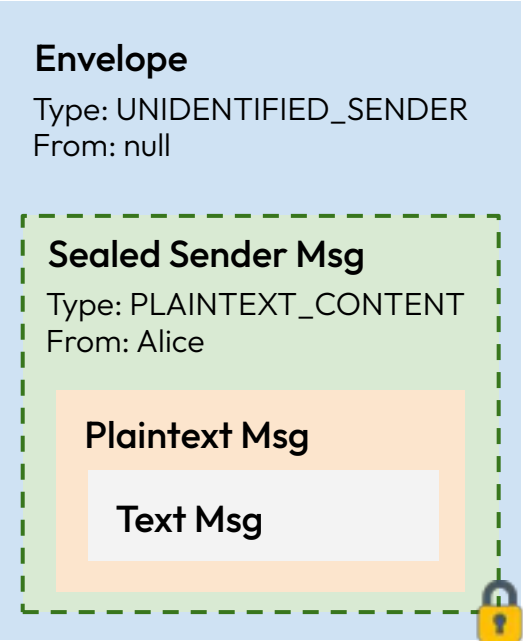
Since there is no authentication, Plaintext messages containing Text messages cannot be accepted. Usually they only carry DecryptionError messages.

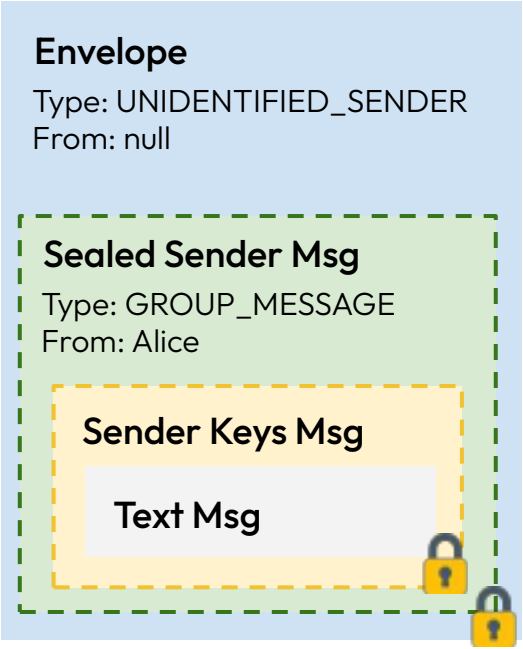

Sealed Sender Msg
Type: WHISPER_MESSAGE
From: Alice
Double Ratchet Msg
Text Msg

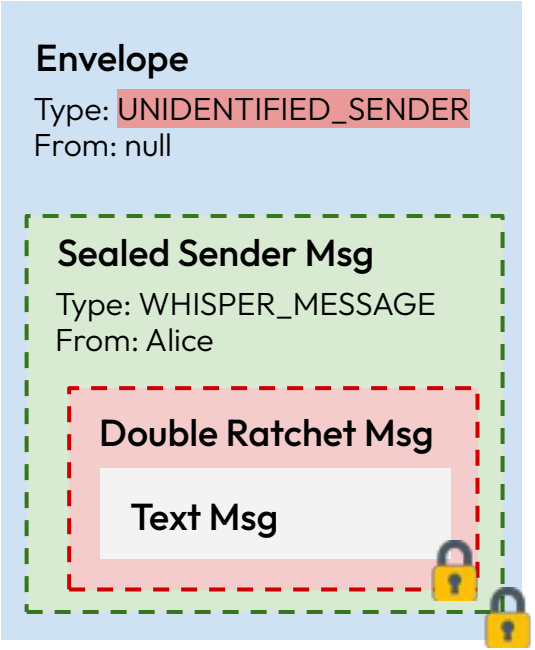

Sealed Sender Msg
Type: PLAINTEXT_CONTENT
From: Alice
Plaintext Msg
Text Msg

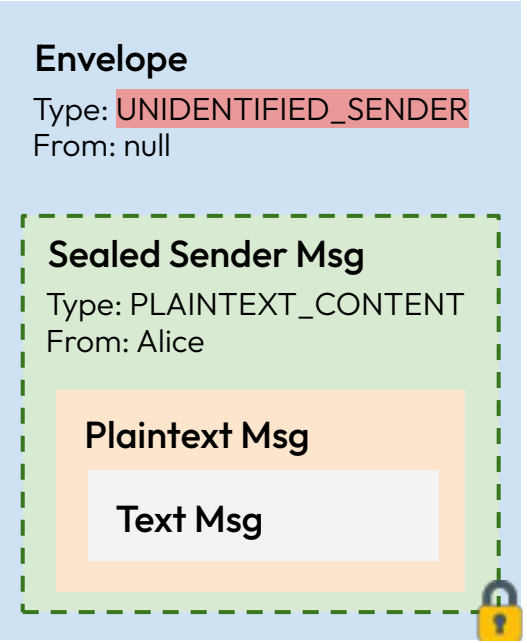

Sealed Sender Msg
Type: GROUP_MESSAGE
From: Alice
Sender Keys Msg
Text Msg

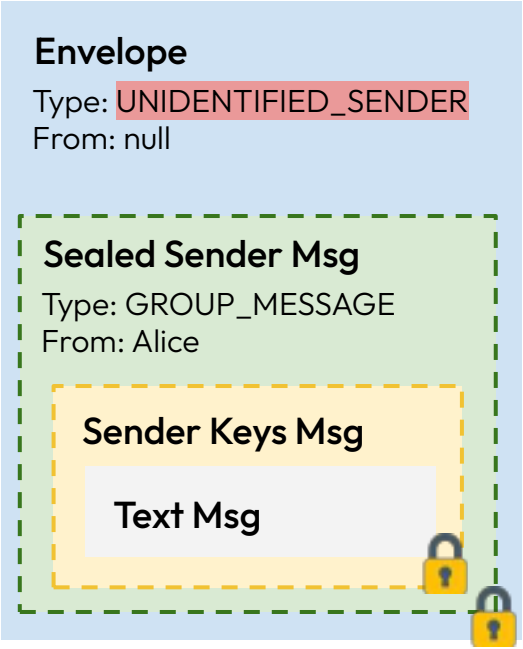

Envelope
Type: UNIDENTIFIED_SENDER
From: null
Sealed Sender Msg
Type: WHISPER_MESSAGE
From: Alice
Double Ratchet Msg
Text Msg


Envelope
Type: UNIDENTIFIED_SENDER
From: null
Sealed Sender Msg
Type: PLAINTEXT_CONTENT
From: Alice
Plaintext Msg
Text Msg

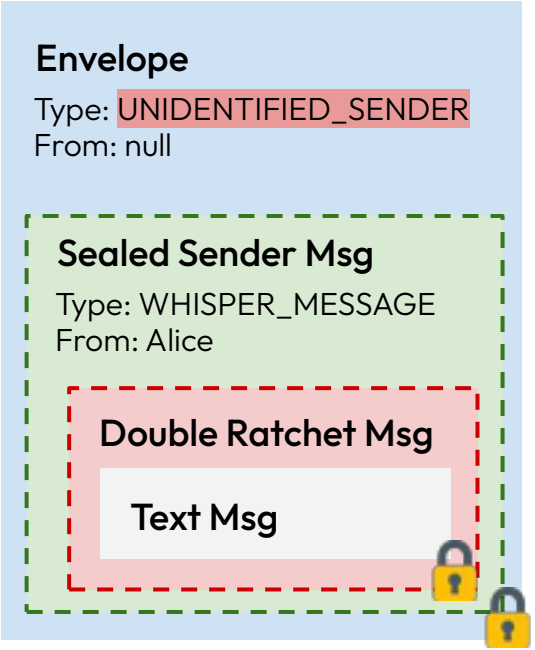

Envelope
Type: UNIDENTIFIED_SENDER
From: null
Sealed Sender Msg
Type: GROUP_MESSAGE
From: Alice
Sender Keys Msg
Text Msg

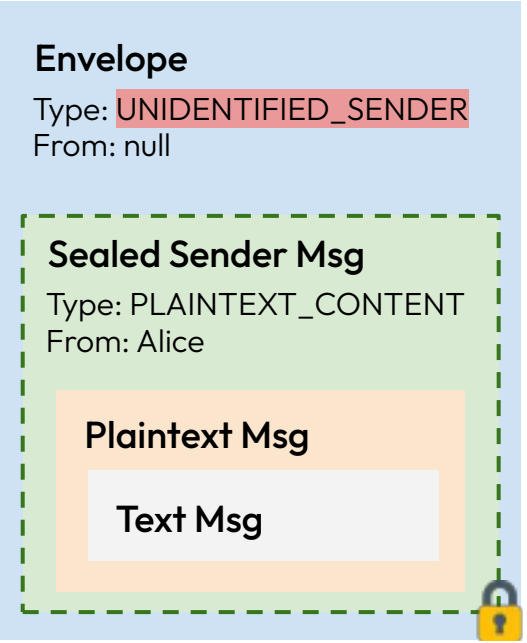


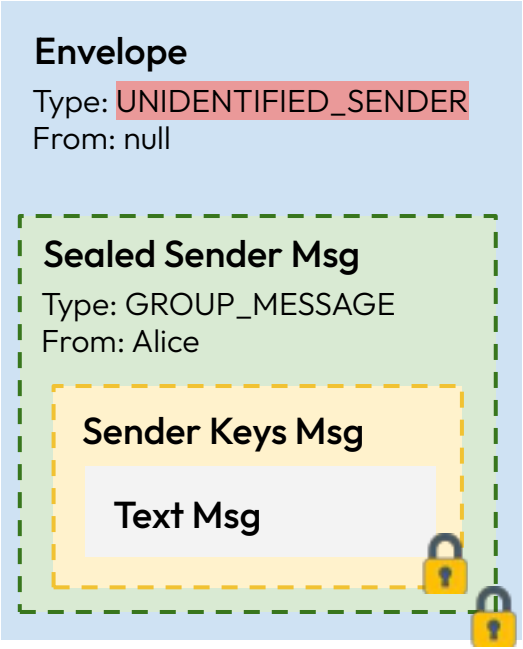




On Android validation is done by looking **only** at the Envelope header



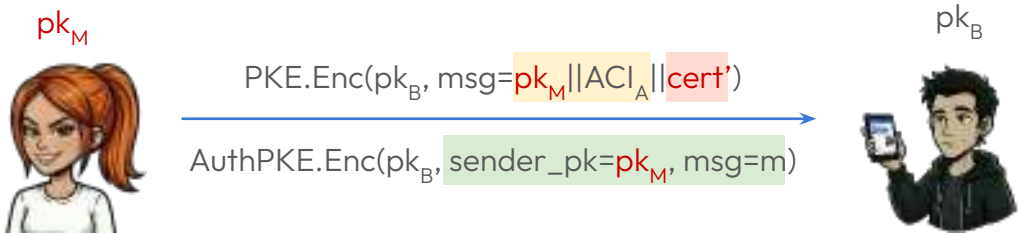




On Android validation is done by looking **only** at the Envelope header

All these messages are accepted by an Android Signal client!

Combining the Two Issues



The server can send messages as any user…

Combining the Two Issues

pk_M

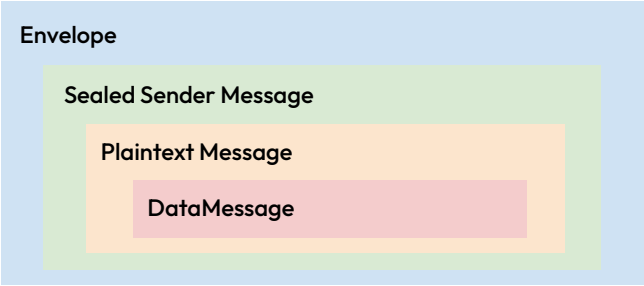
PKE.Enc(pk_B, msg=pk_M||ACI_A||cert')

AuthPKE.Enc(pk_B, sender_pk=pk_M, msg=m)

pk_B

The server can send messages as any user…

… and send plaintext content, bypassing authentication



More About the Attack

- Affected only Android
- Fixed one week after disclosure
- Patched by performing validation on Sealed Sender-wrapped Plaintext messages

What went wrong?

What went wrong?

Despite years of intense academic attention on Signal, these attacks lay undetected

What went wrong?

Despite years of intense academic attention on Signal, these attacks lay undetected

System-level cryptographic security is still a blind spot for the community

What went wrong?

Despite years of intense academic attention on Signal, these attacks lay undetected

System-level cryptographic security is still a blind spot for the community

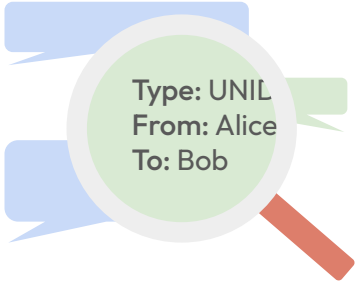
...but do we even have tools to analyze whole systems?

Could we have spotted this earlier?

Formal analyses *could* have spotted these attacks…

Could we have spotted this earlier?

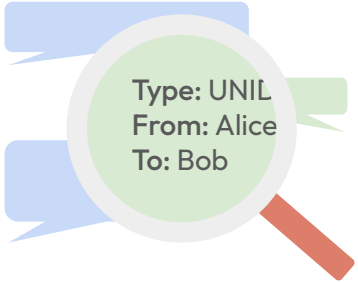
Formal analyses *could* have spotted these attacks...



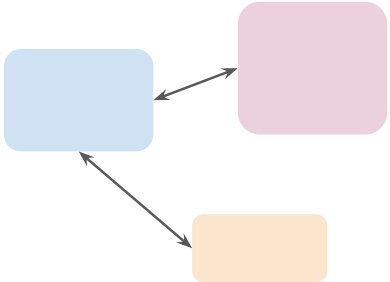
...but only if they captured the system in enough detail

Could we have spotted this earlier?

Formal analyses *could* have spotted these attacks...



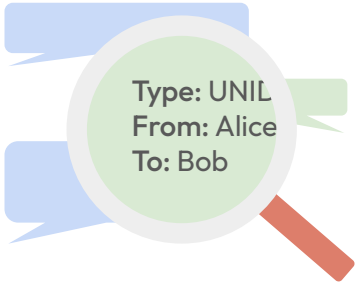
...but only if they captured the system in enough detail



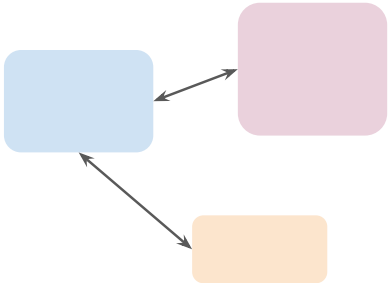
...and only if they captured the interaction between all protocols

Could we have spotted this earlier?

Formal analyses *could* have spotted these attacks...



...but only if they captured the system in enough detail

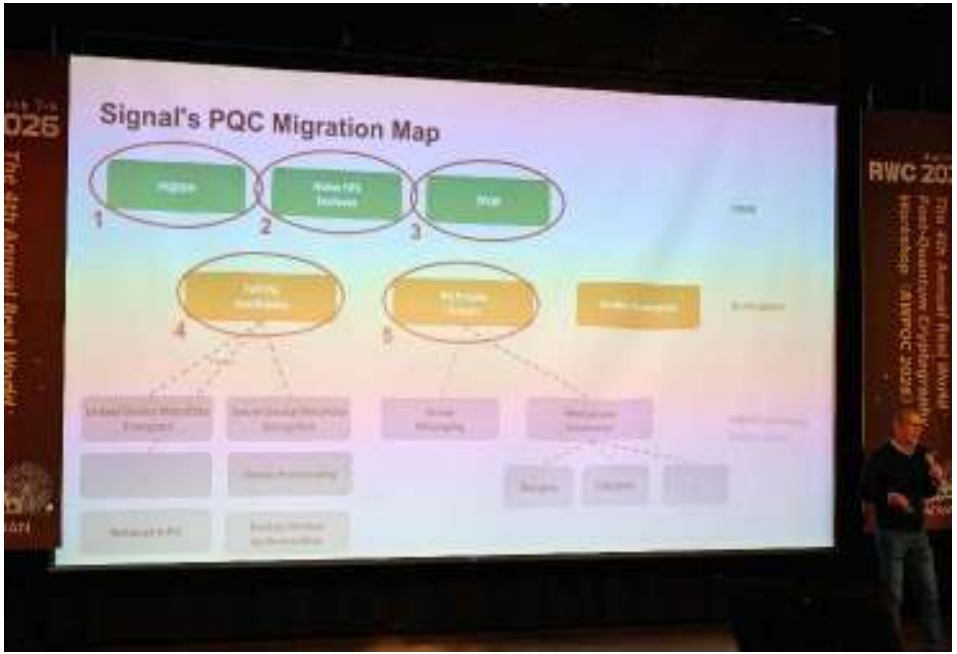


...and only if they captured the interaction between all protocols

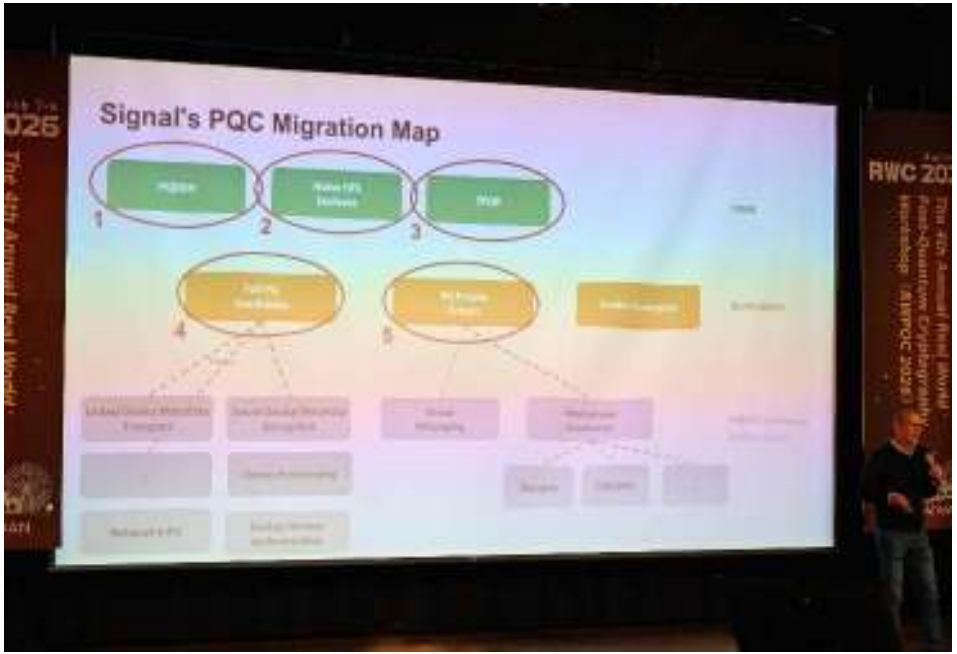
...but our techniques do not seem mature enough to capture this level of complexity yet

Is this complexity good?

Is this complexity good?

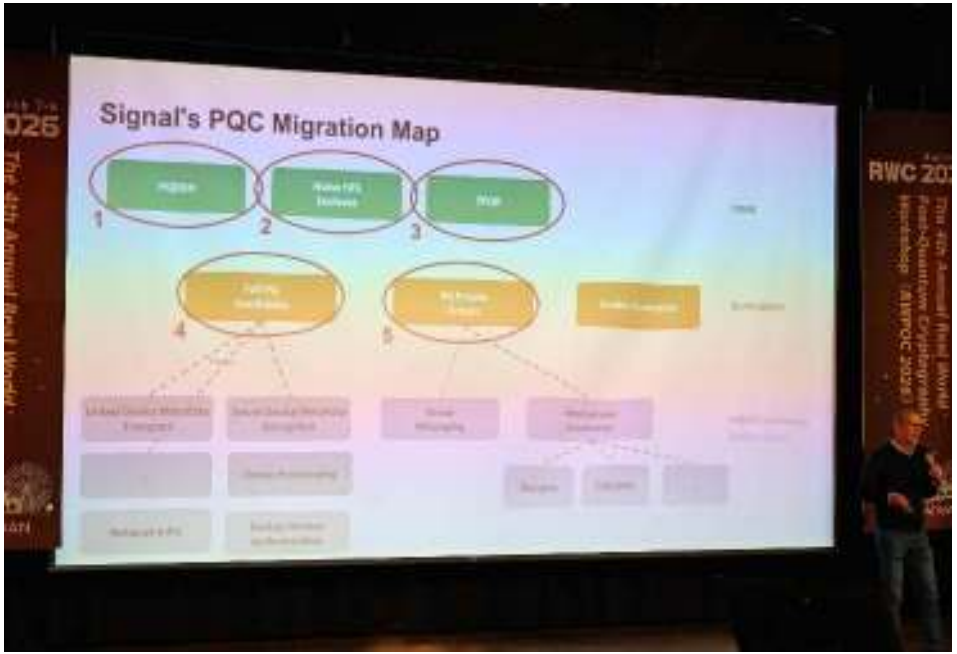


Is this complexity good?



Yes!
The complexity stems from deploying privacy-enhancing features!

Is this complexity good?



Yes!
The complexity stems from deploying privacy-enhancing features!

...but with great features come great responsibilities

Tips for the industry

Open-source code fosters analysis

Tips for the industry

Open-source code fosters analysis

Documentation gives great insights!

Tips for the industry

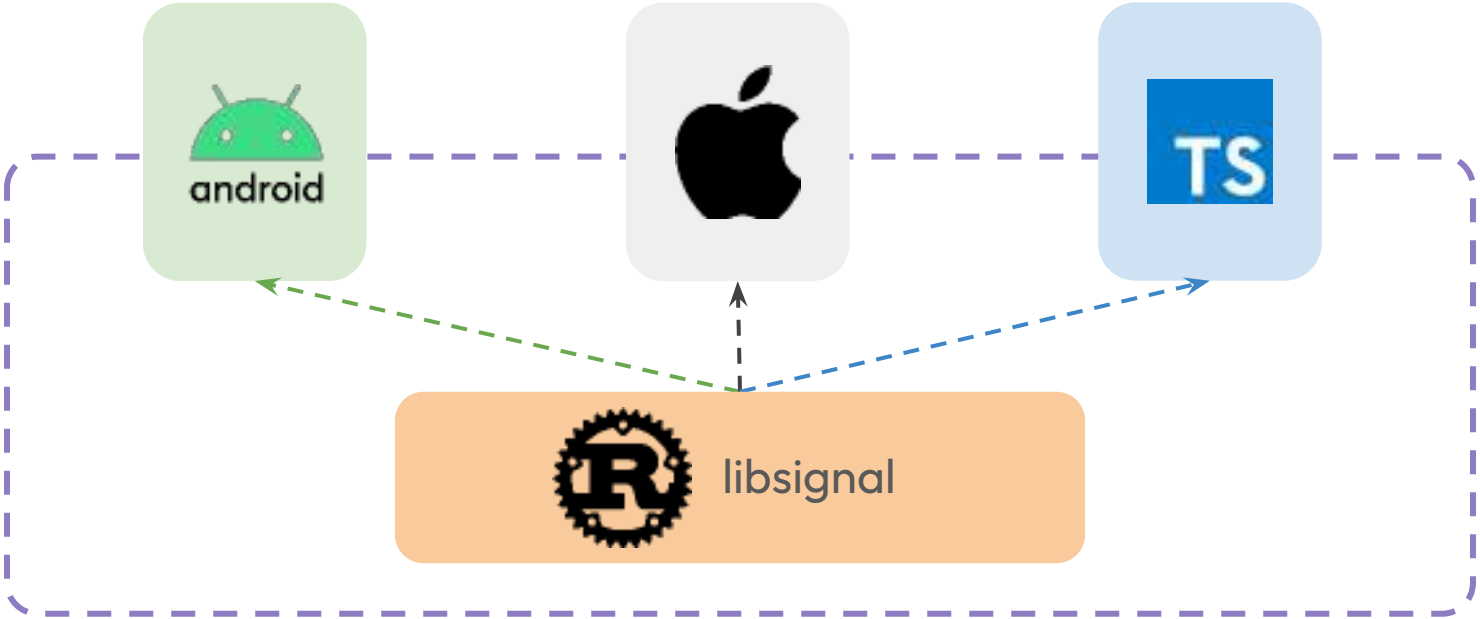
Open-source code fosters analysis

Documentation gives great insights!

We added PNIs/ACIs!

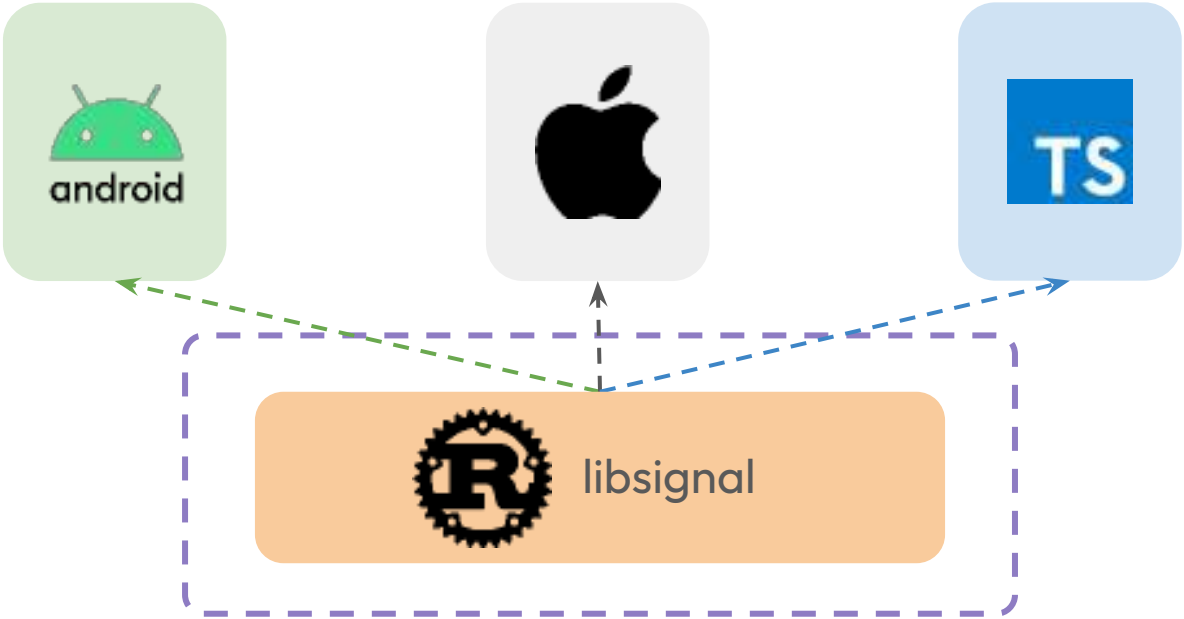
Create a way to communicate “boring” but cryptography-relevant features to the academic community

Libraries for Cryptographic Applications



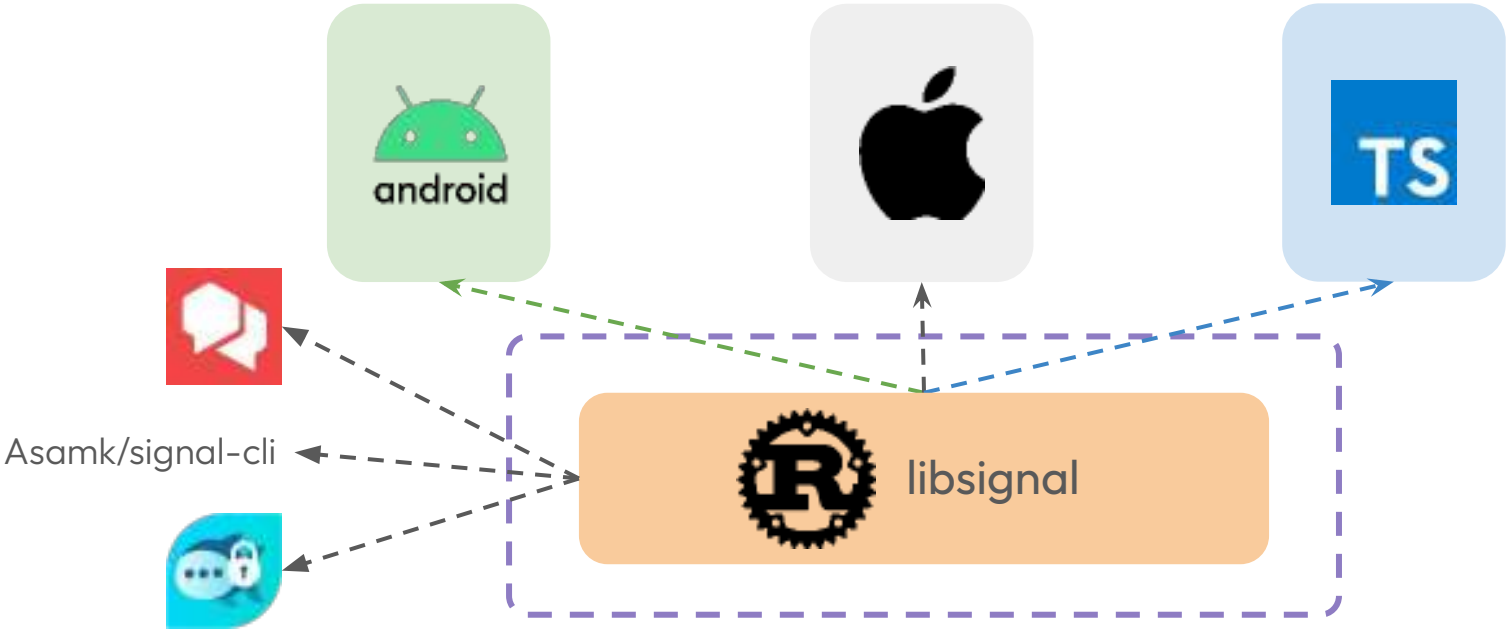
Avoid having cryptographic logic scattered across modules and platforms

Libraries for Cryptographic Applications



Rather, ensure that cryptographic logic is unified in a single (native) library

Libraries for Cryptographic Applications



Rather, ensure that cryptographic logic is unified in a single (native) library

- We analysed Signal in the **malicious server setting**
- We found **two attacks which violate conversation integrity**.
- Attack 1: has pre-conditions and does not affect groups.
- Attack 2: has no pre-conditions and affects one-on-one and group conversations.
- Both attacks were **patched very quickly by the Signal team**, who we thank for the smooth disclosure procedure.

Questions?

noemi.terzo@mpi-sp.org

https://nterzo.sh

kitruong@ethz.ch

https://ktruong.dev

ePrint Paper

Extra Slides

Vulnerability Timeline

2018
Sealed Sender introduced in the “Big Bang” commit (With the first issue already present!)

2021
The Plaintext message type is introduced

2023
Validation is introduced on Plaintext-type messages (but not for Sealed Sender messages)

2026
Our attack is patched by validating Sealed Sender-wrapped Plaintext messages

This vulnerability existed for 5 years!